

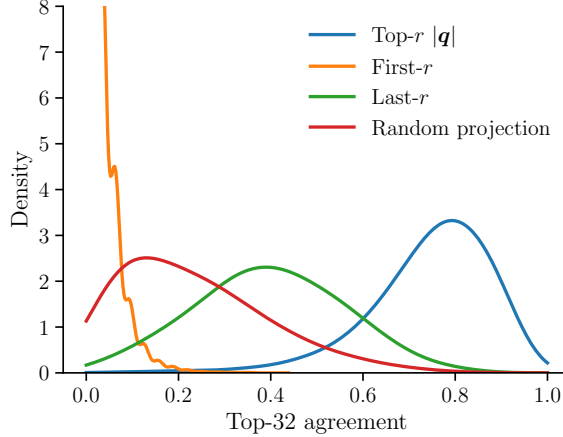


Figure E2: Top- k agreement between approximate and true scores (Llama 2 7B, measured over 40 SQuAD examples). Top- k agreement is the proportion of the top- k positions that are correctly predicted by an approximated softmax, using a projection of $\mathbf{q}$, either component-wise or a random low-rank projection.

F. Benchmarking Detail

Benchmarking code is made available from:

<https://github.com/graphcore-research/llm-inference-research/tree/2024-05-sparq>.

IPU measurements We tested custom fully-fused Poplar implementations of both dense attention and SparQ Attention, compiled using Poplar SDK 3.3.0+1403. On initialisation, we fill large $\mathbf{K}$ and $\mathbf{V}$ tensors with values $\sim N(0, 1)$ in streaming memory. On each benchmarking (outer) iteration, we first randomise the contents of a $\mathbf{q}$ in local memory, then perform multiple inner repeats of the attention op being profiled. We use 4 inner repeats for dense attention, otherwise 1024/batch_size, chosen because dense attention is much slower, and we swept a wide range of settings. We ran an outer loop of 2 warm-up iterations followed by 10 timed iterations, reporting the mean and standard error. The sweep covered $S \in [1024, 2048, \dots, 65536]$, batch_size $\in [1, 4, 16, 64]$, SparQ Attention $r \in [16, 32, 64]$ and $k \in [64, 128, 256, 512]$.

GPU measurements All experiments use PyTorch 2.1.2+cu121 on Ubuntu AWS instances. To set up the experiment, we initialise the large $\mathbf{K}$ and $\mathbf{V}$ tensors with values $\sim N(0, 1)$. On each step, we draw $\mathbf{q} \sim N(0, 1)$, run `torch.cuda.synchronize` before starting a host-side wall-clock timer, run the op, and synchronize again before stopping the timer. We run 20 warm-up iterations followed by 200 timed iterations, reporting mean and standard error. For dense baseline implementations, we tested a vanilla PyTorch implementation, with/without `torch.compile` and `torch.nn.functional.scaled_dot_product_attention`, selecting each backend (math, flash, mem_efficient) manually. For SparQ Attention implementations, we tested vanilla PyTorch (lightly hand-optimised from Appendix B), with/without `torch.compile`. We also toggled fused gather-matmul kernels written in Triton, and whether $\mathbf{K}$ was stored twice in S -contiguous (for **Step 1**) and d_h -contiguous (for **Step 2**) layouts, or only once in d_h -contiguous layout. We tested $S \in [1024, 2048, 4096, 8192, 16384]$, batch_size $\in [1, 4, 16, 64]$, SparQ Attention $r \in [16, 32, 64]$ and $k \in [64, 128, 256, 512]$.

Additional results In addition to the headline results shared in Section 6 and Figure 8, we give an aggregate picture of the trends in Figure F1. Since the number and dimension of heads is fixed, the x-axis is proportional to the size of the input tensors. On IPU (M2000), strong speedups are available across a range of input sizes, principally depending on r , but also on k (not shown). On GPU, sufficient input size is required to observe a speedup over the dense baseline, with the more bandwidth-limited A10G reaching speedups sooner. While part of this effect can be linked to the fundamental additional complexity of SparQ Attention, we anticipate that small input sizes could be accelerated considerably with additional kernel fusion. With an appropriate limit to sequence length, SparQ Attention could even be fused into a single CUDA kernel.

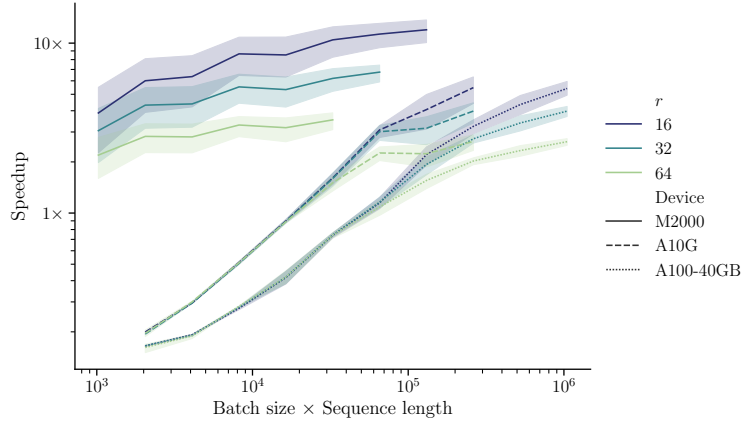


Figure F1: SparQ speedup over the dense baseline, across a range of batch size (1-64), sequence length (1024-65536) and k (64-512), for different devices. We note that for both GPUs, the number of KV elements is a limiting factor for the achieved speedup, and that this could be improved by writing a fully fused SparQ Attention kernel.

Storing K twice One limitation of a theoretical model of data transfer is that it does not account for the granularity of memory access. Since the K matrix is indexed on different axes in **Step 1** and **Step 2** of SparQ Attention, a naive implementation would fetch non-contiguous elements in one of the two steps. To mitigate this, we propose storing K twice, once in S -major format and once in d_h -major format. This increases KV cache memory usage by 50%, but uses only a small amount of extra bandwidth to write k twice. This extra write is non-contiguous, but small, so should not form a bottleneck.

G. Methodology

We provide a comprehensive description of our experimental setup for reference in Table G1.

Baselines We use our own implementation of H₂O (Zhang et al., 2023), which differs from the authors’ implementation in that it uses a fixed cache size k , rather than a fixed proportion of the current sequence length. To validate that these implementations are sufficiently similar, we ran their implementation through our harness on a small model and sample size. On SQuAD 1-shot, with Pythia-1.4B, using $k = 256$, $l = 64$, our implementation was correct for 60 of 200 examples, theirs for 57 (the dense baseline achieved 74). Perhaps more importantly, we found that of the 79 times that either output differed from dense, 41 occurrences showed a 20-character prefix match between our implementation and theirs. The fact that the two implementations often generate the same errors (despite minor implementation differences) reassures us that our results should be a fair representation of H₂O.

Compression ratio We define the compression ratio as the ratio of attention data transfers required for the sparse technique and the dense data transfers. Similarly as we have derived the transfers for SparQ Attention in Section 4, we can define the transfers required for each baseline technique:

$$\begin{aligned}\mathcal{M}_{\text{H}_2\text{O}} &= 2k d_h + 2d_h + 2S \\ \mathcal{M}_{\text{LM}_\infty} &= 2k d_h + 2d_h \\ \mathcal{M}_{\text{FlexGen}} &= S d_h + k d_h + 2d_h\end{aligned}$$

and $\mathcal{M}_{\text{dense}} = 2S d_h + 2d_h$. For each technique, the compression ratio is then $\mathcal{M}_{\text{technique}}/\mathcal{M}_{\text{dense}}$.

Models	Llama 2	7B ($d_h = 128$, Max $S = 4096$, $g = 1$) 13B ($d_h = 128$, Max $S = 4096$, $g = 1$)
	Llama 3	8B ($d_h = 128$, Max $S = 8192$, $g = 4$)
	Mistral	7B ($d_h = 128$, Max $S = 8192$, $g = 4$)
	Gemma	7B ($d_h = 256$, Max $S = 8192$, $g = 1$)
	Pythia	1.4B ($d_h = 128$, Max $S = 2048$, $g = 1$) 2.8B ($d_h = 80$, Max $S = 2048$, $g = 1$) 6.9B ($d_h = 128$, Max $S = 2048$, $g = 1$)
Tasks	Question Answering	SQuAD 1-shot (4000 samples) TriviaQA 0-shot (2992 samples)
	Summarisation	CNN/DailyMail 0-shot (500 samples)
	Language Modelling	WikiText-103 LM (500 samples)
	Artificial	Repetition (1000 samples)
Baselines	H ₂ O	keep $(k - l)$ tokens with highest $\text{score}(n) = \sum_i s_{in}$ and the most recent $l = k/4$ $k \in \{192, 256, 384, 512, 768\}$
	LM-Infinite	take the first 16 tokens, and most recent $k - 16$ $k \in \{192, 256, 384, 512, 768\}$
	FlexGen	take top- k tokens using exact attention scores $k \in \{2, 8, 32, 128, 256\}$
SparQ Attention	Rank r	$\{8, 16, 32, 64\}$
	Number of values k	128
	Local window l	$k/4$

Table G1: Experimental setup.